

Dravika: Evaluation Evidence Kit

Every asset the deck can use, organised by the five SIH scoring parameters. Each option is tagged **READY** (measured, generated from the repo, drop-in) or **ROADMAP** (planned, say it as future work, never as a result). Images live in `tools/deck/assets/` ; every number regenerates from `npm run bench` , `npm run bench:latency` and `npm run demo:deblur` .

Inventory: 5 stat-tile sets · 6 charts · 4 prototype screenshots · 3 attack images · 9 rendered flowcharts · 4 benchmark tables · 235 automated tests behind the numbers.

Rule of use: one hero asset per slide plus one table or stat row. Do not stack three charts on one slide.

1. Accuracy of visual context from screen

25%

Does the agent truly understand what is on screen: elements, roles, states, geometry?

IMAGE

READY

Option A: the scene-graph overlay (hero asset for this metric)

Jan Seva Portal
Citizen Services - Government of India

Scene graph: 11 regions, ~15 ms
DOM = accessibility walk, no models
green filled / blue empty/actionable
red unexplained pixels (fail closed)

#1 heading
Address Verification - Step 2 of 3

#2 text
Application reference: 78451238014

Please verify your registered details and complete the address section below.

#3 image - unexplained
The photograph and identity details below were fetched from your profile. Contact the helpdesk on 1800-000-000 if anything is incorrect.

Applicant photo

#4 textinput - required - filled
Ramesh Kumar

#5 textinput - required - filled
9999 4105 7058

#6 textinput - required - filled
ramesh.kumar@gmail.com

#7 textinput - required - filled
+91 98765 43210

#8 textinput - required - empty

#9 textinput - required - empty

#10 textinput - required - empty

#11 button - disabled - state-changing
Proceed to Verification

Fill all required fields to continue.

evidence-scenegrph.png: how Dravika sees the demo form. 11 regions with ids, roles and states from a ~15 ms DOM + accessibility walk; the photo is flagged unexplained (fail closed).

Say this: Structure explains the whole form in 15 milliseconds with zero models. Pixels are only consulted where structure cannot vouch, and anything unexplained is masked, never guessed.

DIAGRAM

READY

Option B: the re-grounding loop

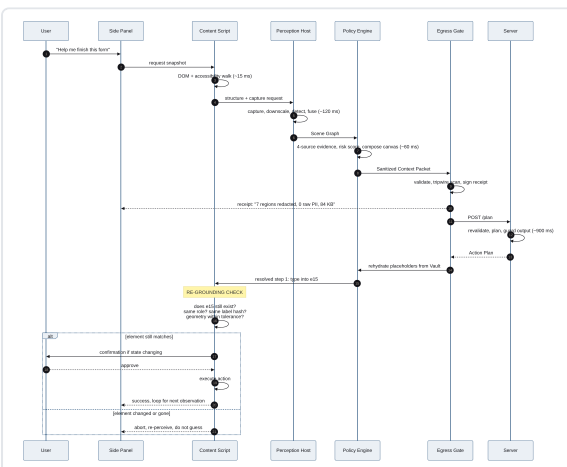


diagram-loop.png: before any action, the element is re-verified (exists, same role and label, geometry within tolerance). Page changed means abort and re-perceive.

Say this: Accuracy is not just seeing correctly once; it is refusing to act on a stale view.

TABLE

READY

Option C: perception mechanisms, each pinned by tests

Mechanism	Evidence
DOM + accessibility walk with roles, labels, states	~15 ms, measured; happy-dom test suite
Open shadow-root traversal, hidden/off-screen filtering	unit tested
Vision-to-DOM fusion (IoU + centroid + role compatibility)	15 fusion tests, DPR-safe coordinates
On-device face pass on the same frame the composer redacts	54.4 ms measured, real ONNX on Linux WASM
Unexplained pixels fail closed	policy tests + red team
Re-grounding before every action	executor tests

STAT

ROADMAP

Option D: explained-area fraction and grounding accuracy

The report defines *explained-area fraction* (share of visible pixels accounted for by structure) and ScreenSpot-style grounding accuracy on a pixel-annotated corpus. The functions ship (`fusion.explainedAreaFraction`); the pixel-level corpus does not yet. Present as the metric we will publish, not a number we have.

2. Recall and precision for detection of sensitive / PII data

20%

Catch everything sensitive without crying wolf on lookalikes.

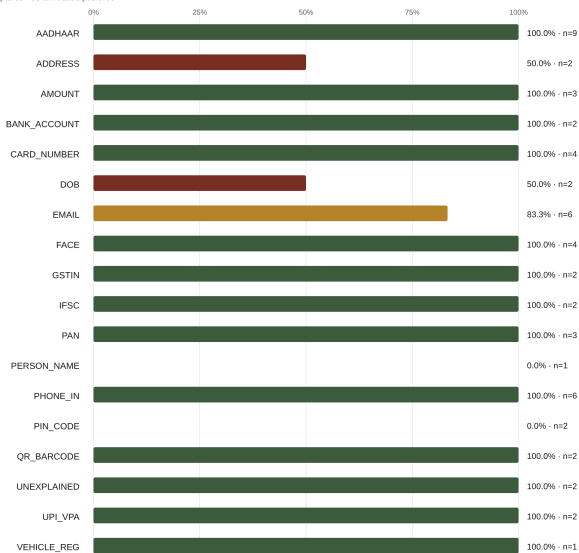
CHART

READY

Option A: recall by class with the gaps annotated (hero asset)

Shield detection recall by PII class

RedactBench-Web · 18 synthetic captures · 55 annotated positives



Misses are published: free-text address, obfuscated email, and context-dependent DOB.

chart-recall.png: 18 classes on the adversarial corpus; every gap named in place and pinned by a CI gate.

Say this: We attacked our own benchmark: unicode dashes, spaced-digit evasions, Hindi labels, table splits. It found a real detector bug, we fixed it, and the recall that survived is 89.1% with every remaining gap published.

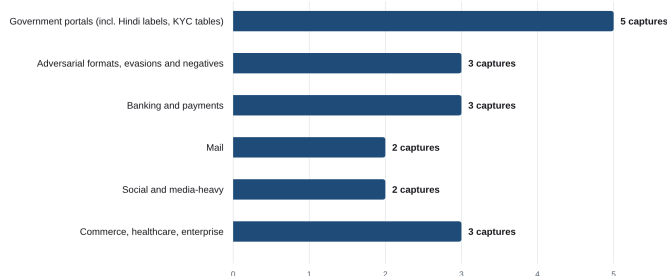
CHART

READY

Option B: corpus composition (the demographics)

RedactBench-Web corpus composition

18 captures · 55 annotated positives · 12 declared lookalike negatives · all personas synthetic (UIDAI test ranges)



Negatives make precision measurable: order ids, toll-free helplines, appointment dates, employee ids, en-dash lookalikes, Luhn-fail cards and spaced order ids must NOT be masked, and are not.

evidence-corpus.png: 18 captures, 55 positives, 12 declared negatives, all synthetic personas.

TABLE

READY

Option C: headline detection table

Mode	Micro precision	Micro recall	Invariant leaks	False alarms
Shield	100%	89.1%	0	0
Fortress	96.3%	94.5%	0	1 (appointment date)

Checksummed classes (Aadhaar, PAN, GSTIN, cards, IFSC): 100/100. Support shown per bar in Option A.

TABLE

READY

Option D: the negatives table (what must NOT be masked)

Lookalike	Why it stays visible
Order id 784512369014	12 digits, fails Verhoeff
Ref 8473–2619–0459 (en-dash)	fails Verhoeff
Card-shaped 4539 1488 0343 6468	fails Luhn
Spaced run 7 8 4 5 1 2 3 6 9 0 1 4	evasion recognizer checksum-rejects it
Toll-free 1800-000-000, appointment dates, employee ids	not personal data

Say this: Precision is only real if something is allowed to stay visible. These are on screen in the demo, unmasked, on purpose.

CHART

ROADMAP

Option E: ablation chart (regex only vs +checksums vs +context vs +cross-region)

A rising ablation bar chart proving each detector layer earns its place. About a day of work on the existing registry; strong add if time allows.

3. Precision of redaction

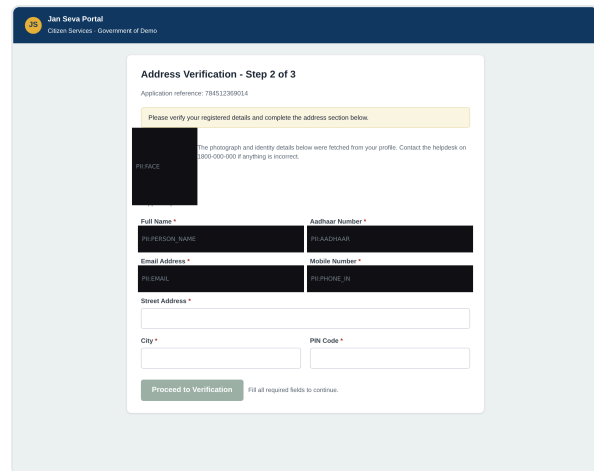
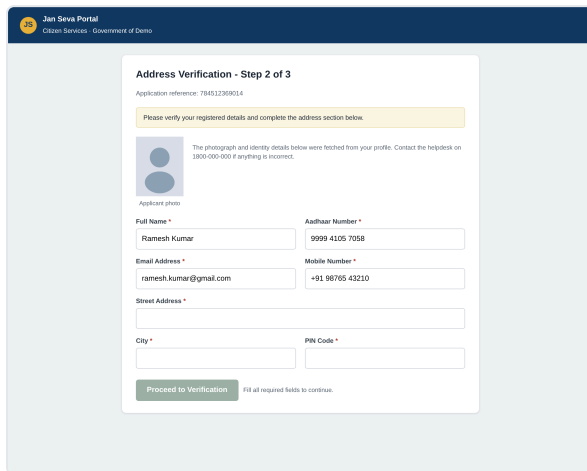
20%

Masks in the right place, tight, and genuinely unrecoverable.

IMAGE

READY

Option A: the split screen (hero asset)



proto-form.png vs proto-form-sanitized.png: flat fills with category stamps at real element geometry; the application reference and helpline stay readable (precision, not paranoia).

Say this: What you see on the right is everything the server knows. It has never seen anything else.

IMAGE

READY

Option B: the blur-recovery attack (the memorable 30 seconds)

4539148803436467

original



blurred far past human readability (sigma 14)



what Dravika sends

Attack result: blur → 16/16 digits recovered (100%). Flat fill → chance level, identical candidate ranking in every cell: zero plaintext information. Reproduce live: `npm run demo:deblur`

Say this: Most teams will blur. Blur is not redaction, and we brought the attack that proves it.

READY

Option C: the fresh-canvas rule and the self-check

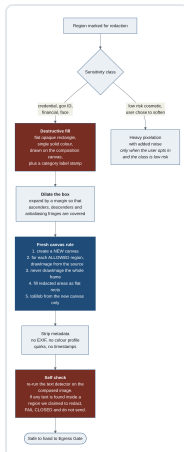


diagram-redaction.png: output starts as fill colour; source pixels are copied only into spans proven outside every dilated redaction. A span bug loses image, never leaks it.

READY

Option D: verification facts

Guarantee	Mechanism
No source pixel inside a claimed redaction	pixel-level self-check, run twice (pre and post stamp)
Antialiased fringes covered	adaptive dilation, ~15% of text height, 2 px floor
Self-check failure cannot leak	frame dropped, structure-only sent; task continues
What was sent is provable	sha256 of the exact payload in the signed receipt
Wireframe mode	schema itself rejects any pixel payload

ROADMAP

Option E: pixel coverage / over-mask / IoU numbers

The scoring functions ship (`bench.scoreRedaction` : coverage, over-mask ratio, mean IoU) and are unit tested; publishing numbers needs the pixel-annotated corpus. Frame as the next benchmark release.

4. Client-side resource utilisation

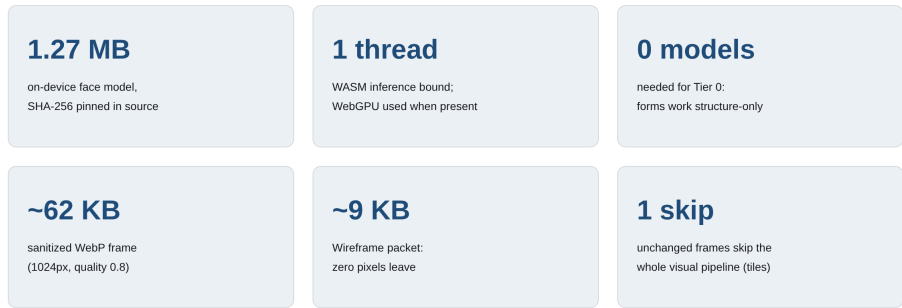
20%

Does it run on an ordinary machine without making the fans scream?

CHART READY Option A: resource footprint tiles (hero asset)

Client-side resource footprint

everything runs in a browser side panel: no GPU required, no server-side vision, no telemetry



Degradation ladder: WebGPU -> single-thread WASM -> structure-only. Every rung ships and is tested; nothing assumes a GPU.

evidence-resource.png

Say this: One 1.27 MB model, one WASM thread, zero GPU required, and Tier 0 needs no models at all. The heavy thinking stays on the server by design.

DIAGRAM READY Option B: the model ladder

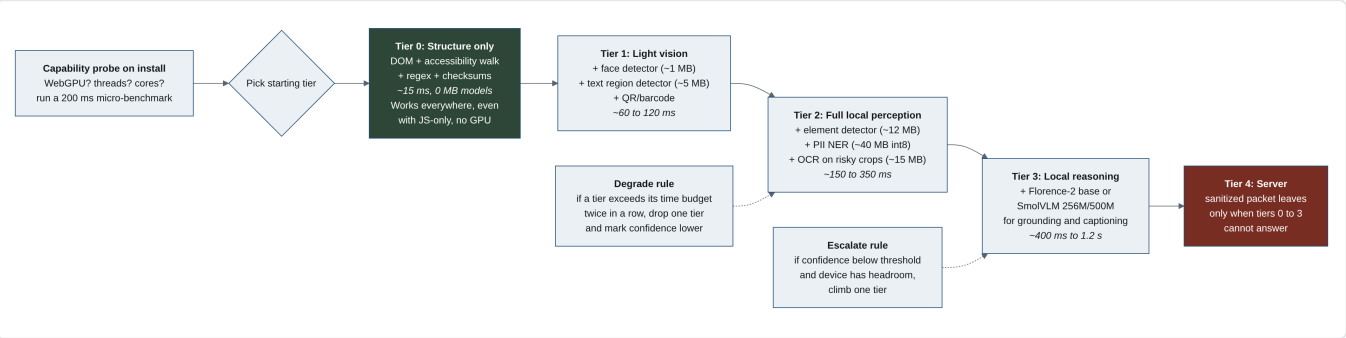


diagram-ladder.png: WebGPU to WASM to structure-only; every rung ships and degrades gracefully.

TABLE READY Option C: measured footprint table

Item	Measured
Face model (only model shipped)	1.27 MB, SHA-256 pinned, provenance is a CI test
Inference CPU cost	54.4 ms on ONE wasm thread (no GPU assumed)
Sanitized frame on the wire	~62 KB WebP at 1024 px; Wireframe packet ~9 KB, zero pixels
Unchanged frames	tile hash (0.37 ms) skips the entire visual pipeline
Whole local pipeline	71.2 ms CPU per frame, single core

TABLE

ROADMAP

Option D: three device classes (low / mid / high)

The report commits to CPU%, heap, battery and host-page FPS impact on three machines including a no-GPU laptop. Not yet run; present as the measurement plan, with the M5 single-thread numbers as the current conservative bound.

5. Overall end-to-end latency of the provided task

15%

How long from ask to done?

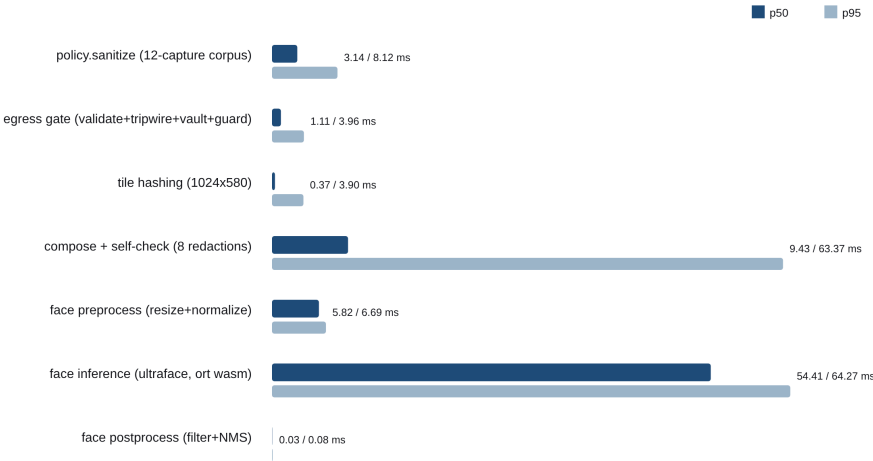
CHART

READY

Option A: the latency waterfall (hero asset)

Local pipeline latency by stage

Measured by 'npm run bench:latency' on linux/x64, Intel(R) Core(TM) i3-6100U CPU @ 2.30GHz, Node v24.15.0. Real code paths: ONNX runs on the single-threaded wasm backend, which is the conservative



Each stage shows p50 (top) and p95 (bottom); single-thread WASM measurement.

chart-latency.png: per-stage p50/p95 on real code paths.

Say this: The entire privacy layer costs 71.2 milliseconds against a 175 millisecond budget. End-to-end latency is dominated by the remote model, exactly as designed.

TABLE

READY

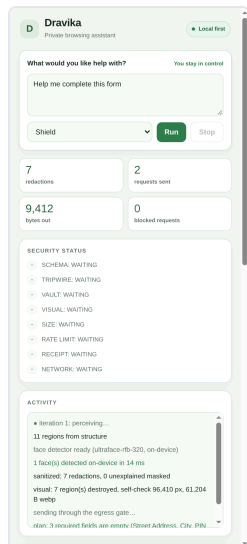
Option B: end-to-end budget vs measured

Segment	Status	Value
Local perceive + sanitize + gate (per frame)	measured	71.2 ms p50 (budget 175 ms)
Policy engine over the full 18-capture corpus	measured	0.33 ms
Network + planner round trip	design budget	~0.8 to 1.8 s, model-dominated
Unchanged-frame iteration	measured mechanism	visual pipeline skipped entirely

IMAGE

READY

Option C: the panel with live timings



proto-panel.png: the activity log prints per-step timings during the demo, so latency is observed by the judges, not claimed.

STAT

ROADMAP

Option D: wall-clock on the 12 benchmark tasks

Task-level end-to-end timings (cold and warm) across the report's 12 task set, per mode. Requires the live-browser run harness; the strongest closing number once measured.

Regenerate everything: `npm run bench (detection), npm run bench:latency (stages), npm run demo:deblur (attack), node tools/deck/gen-assets.mjs + render-charts.mjs + gen-evidence.mjs (images), node tools/deck/build-deck.mjs (deck)`. Run current tests and browser fixtures first: `npm test` and `npm run test:browser`. See docs/IMPLEMENTATION-EVIDENCE.md for exact measured scope and remaining limits.